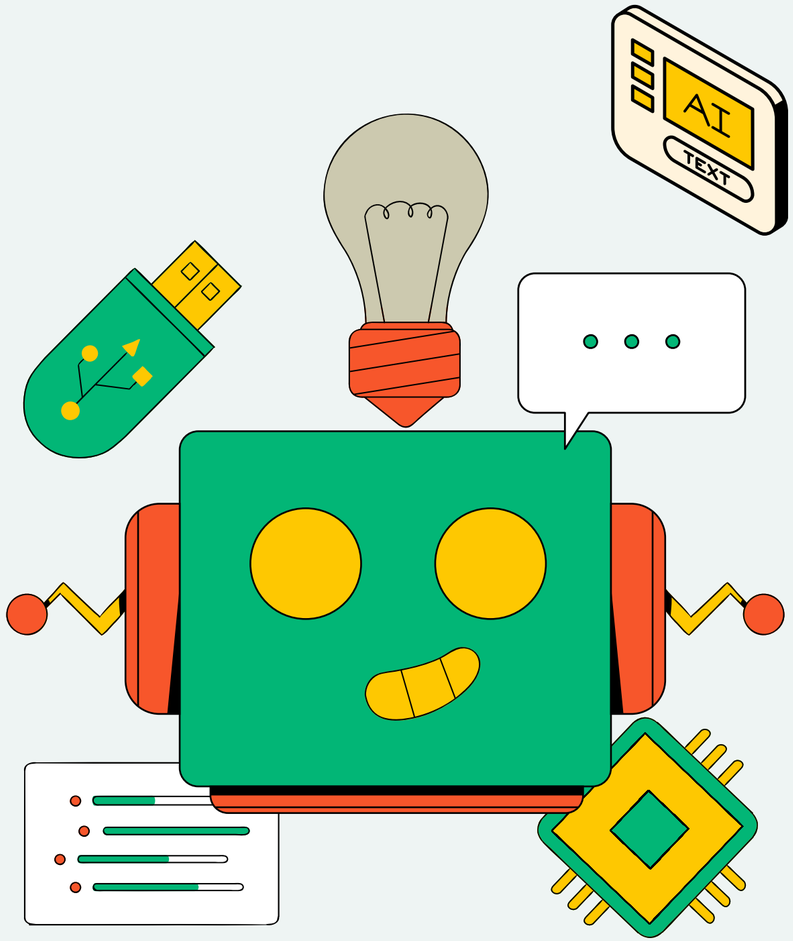
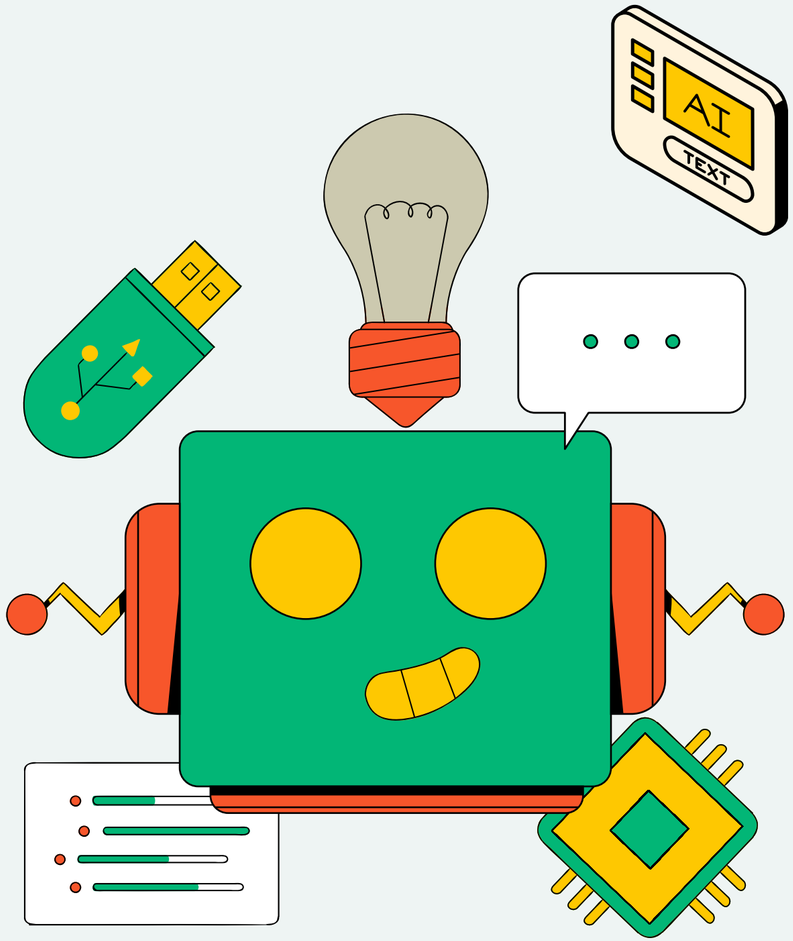


DESIGN OF INTELLIGENT POSITION COTROLLER USING PPO ALGORITHM

MCTA 4362 MACHINE LEARNING

Group Member



- MUHAMMAD MUHAIRIS BIN AZMAN (2114599)
- MUHAMMAD HAFIDZUDDIN HANIF DANIAL BIN NORIZAL (2123651)
- SHAREEN ARAWIE BIN HISHAM (2116943)
- IBRAHIM BIN NASRUM (2116467)

OUTLINE

- Introduction
- Objective
- Problem Statement
- Component Used
- Mathematical Modeling
- Circuit Design
- Evaluation
- Comparison
- Limitation
- Solutions and future work
- Simulation

INTRODUCTION

This project aims to compare a classical PID controller and a PPO-based controller for DC motor position control to evaluate their performance in handling dynamic systems.

OBJECTIVE

To model and simulate a control system using DC motor and potentiometer

To implement Reinforcement Learning algorithms i.e PPO to train an intelligent controller.

To compare the RL controller against classical control methods (PID) using appropriate performance metrics.

PROBLEM STATEMENT

Design a PPO-based controller for it

Compare the performance with a classical PID controller

Demonstrate the strengths and limitations of both approaches

PID

PID Control Logic

- Formula:

$$\text{Output} = K_p \cdot e + K_i \cdot \int e\,dt + K_d \cdot \frac{de}{dt}$$

- PID Coefficients:
  - K_p = 1.8 (proportional gain)
  - K_i = 0.03 (integral gain)
  - K_d = 0.4 (derivative gain)

- Deadband: ±5 units to avoid unnecessary motor movements

PPO

Model: PPO (Proximal Policy Optimization)
Policy: MlpPolicy (multi-layer perceptron neural network)
Environment: env (custo environment)
Verbosity: 1 (prints training info)

Hyperparameters:

- Learning rate: 0.0005
- Steps per update: 1024
- Batch size: 128
- Discount factor (gamma): 0.98 (for future rewards)
- GAE lambda: 0.95 (for advantage estimation)
- Entropy coefficient: 0.0001 (lower value reduces exploration randomness)
- Device: CPU (no GPU)

Training Settings:

- Total timesteps: 200,000 steps of training
- Maximum steps per episode: 500

REWARD SYSTEM

```
reward = - (error / self.max_position) ** 2
        reward -= 0.01 * pwm_norm
        if error < 2.0:
            reward += 2.0

        reward = np.clip(reward, -1.0, 2.0)

        self.step_count += 1
        done = error < 2.0 or self.step_count >= self.max_steps
        return self._get_obs(), reward, done, False, {}
```

METHODOLOGY OF SELECTED MODEL

SYSTEM SELECTION

- Select a dynamic plant suitable for control (e.g., DC motor)
- Define control objective: tracking a reference position

MODEL DEVELOPMENT

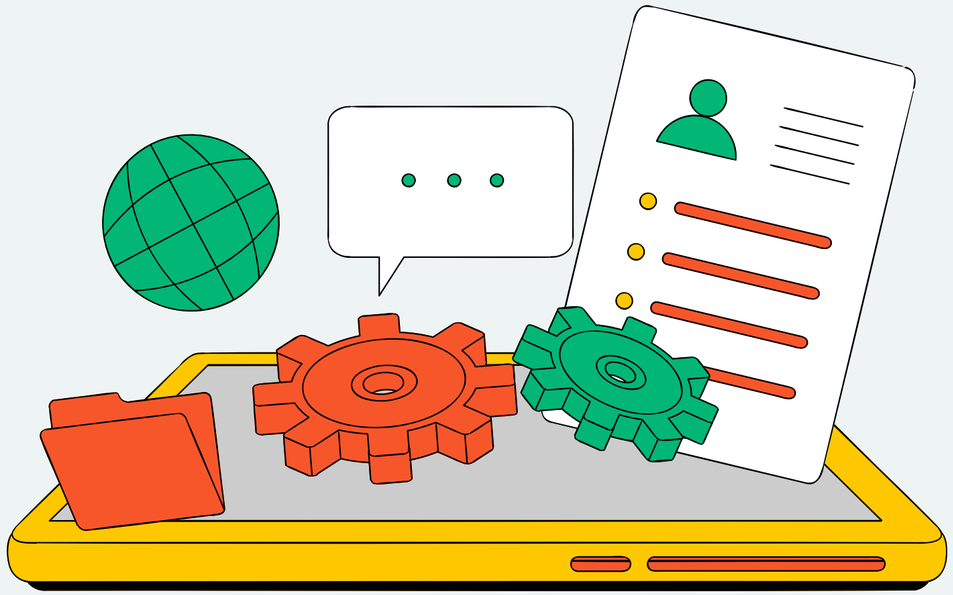
- Selection of appropriate machine learning algorithms
- Model architecture and hyperparameter tuning
- Training process and validation

MODEL EVALUATION

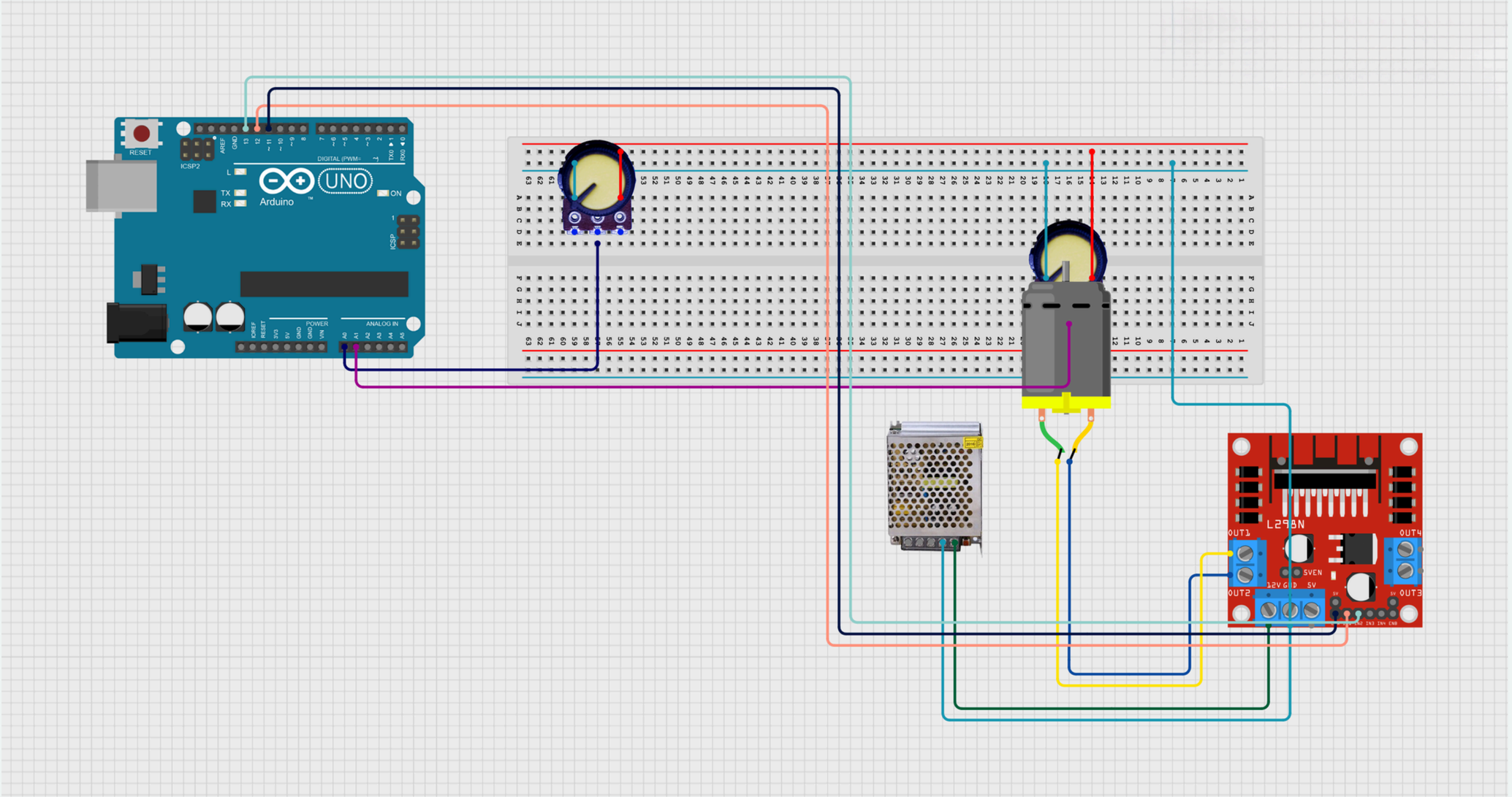
- Cross-validation and testing on validation datasets
- Evaluate performance using Root Mean Square Error (RMSE)

COMPONENTS USED

- Arduino Uno
- Motor Driver (L298N)
- DC Motor
- Power Suply 12 V
- Potential meter
- Breadboard

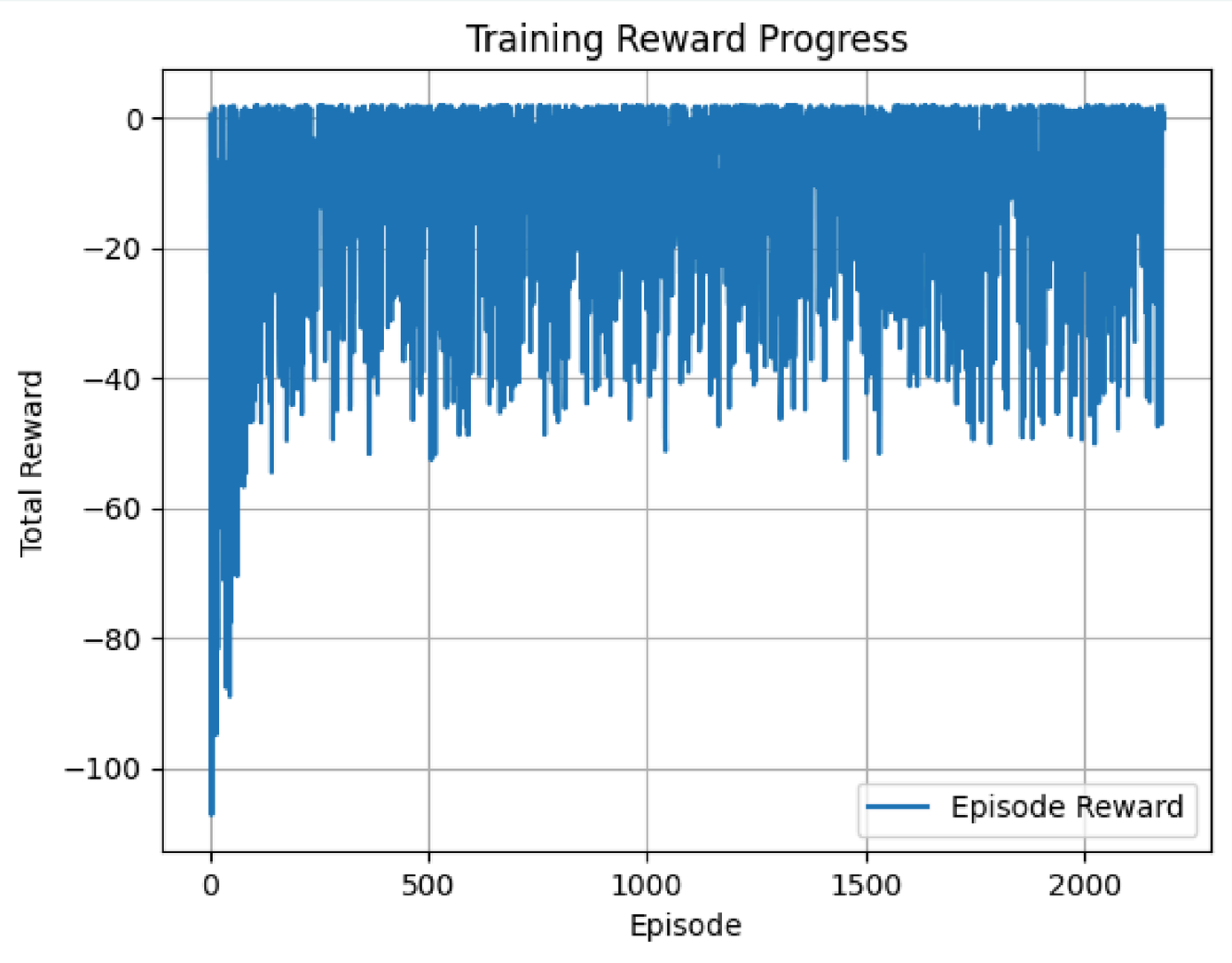


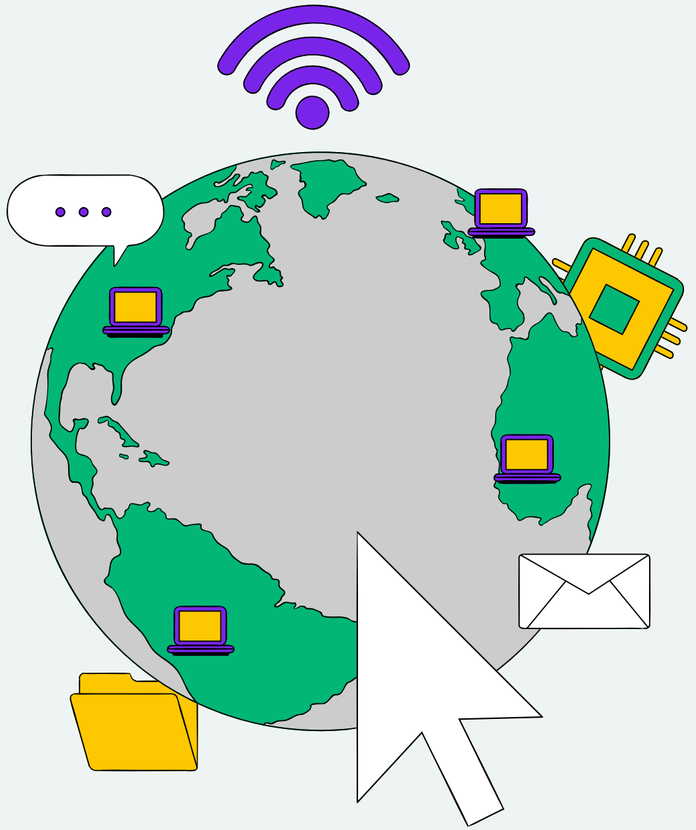
CIRCUIT DESIGN



SIMULATION

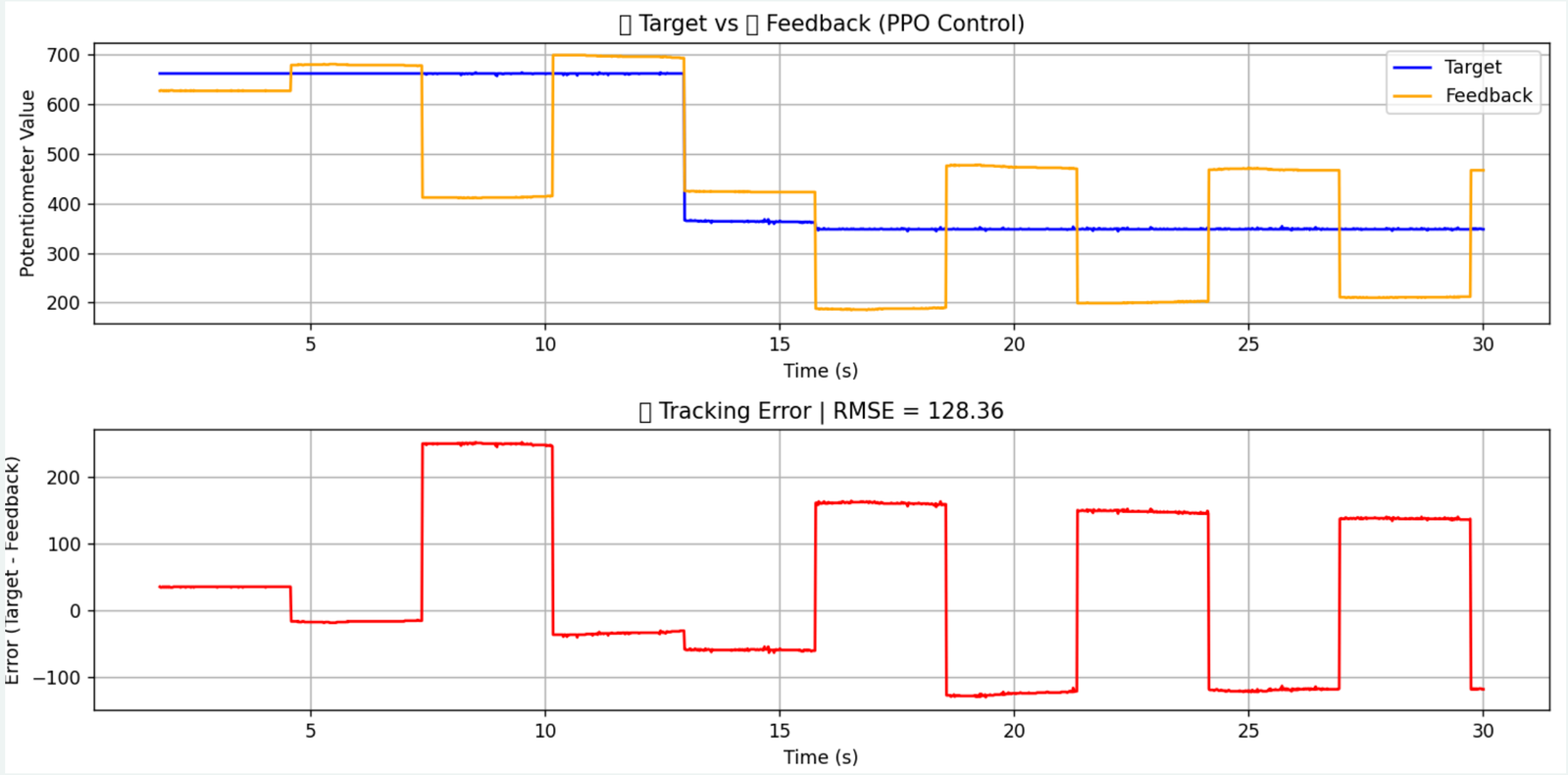
REWARD RESULT

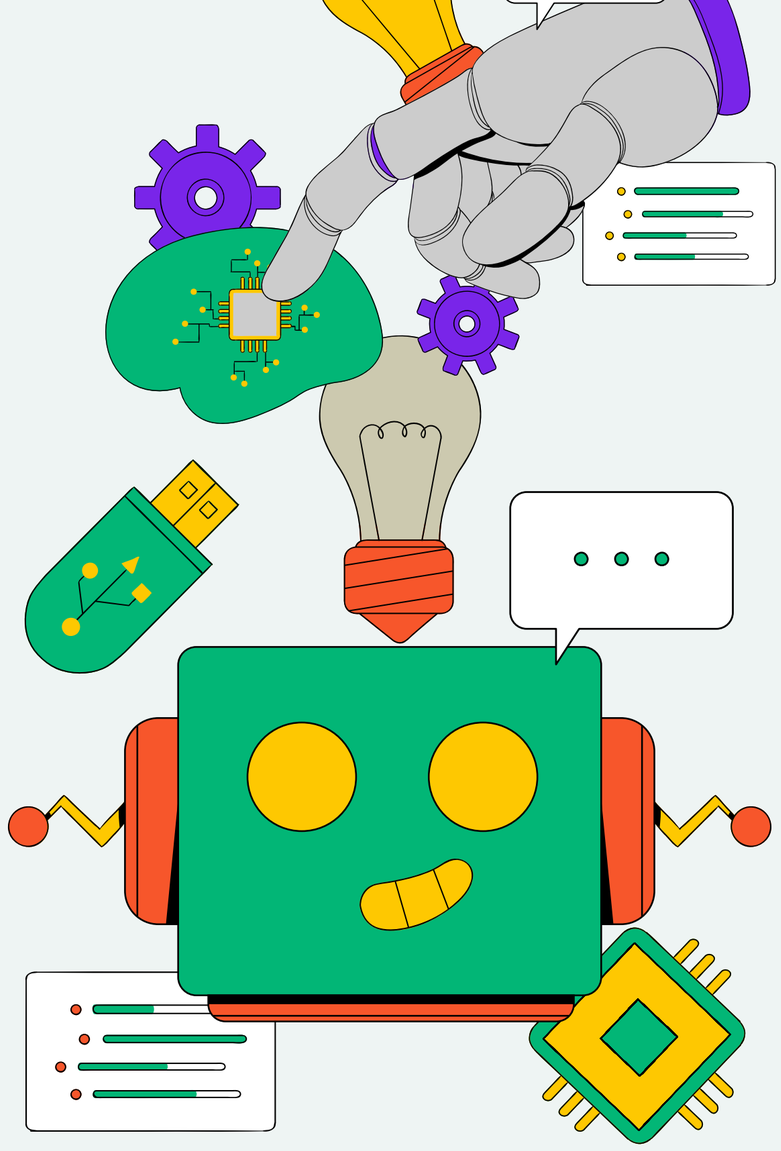




EVALUATION

PPO CONTROL INFERENCE

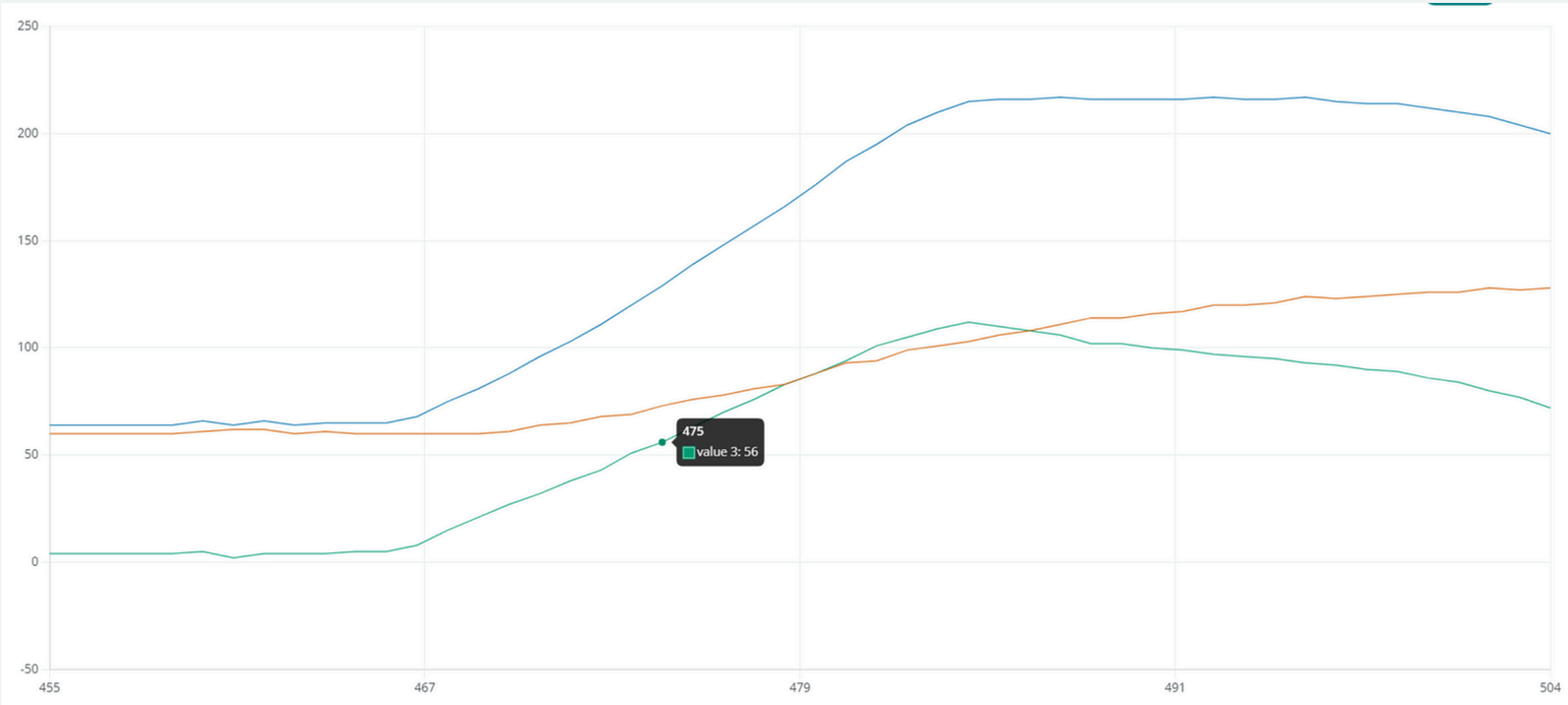




RMSE = 128.36

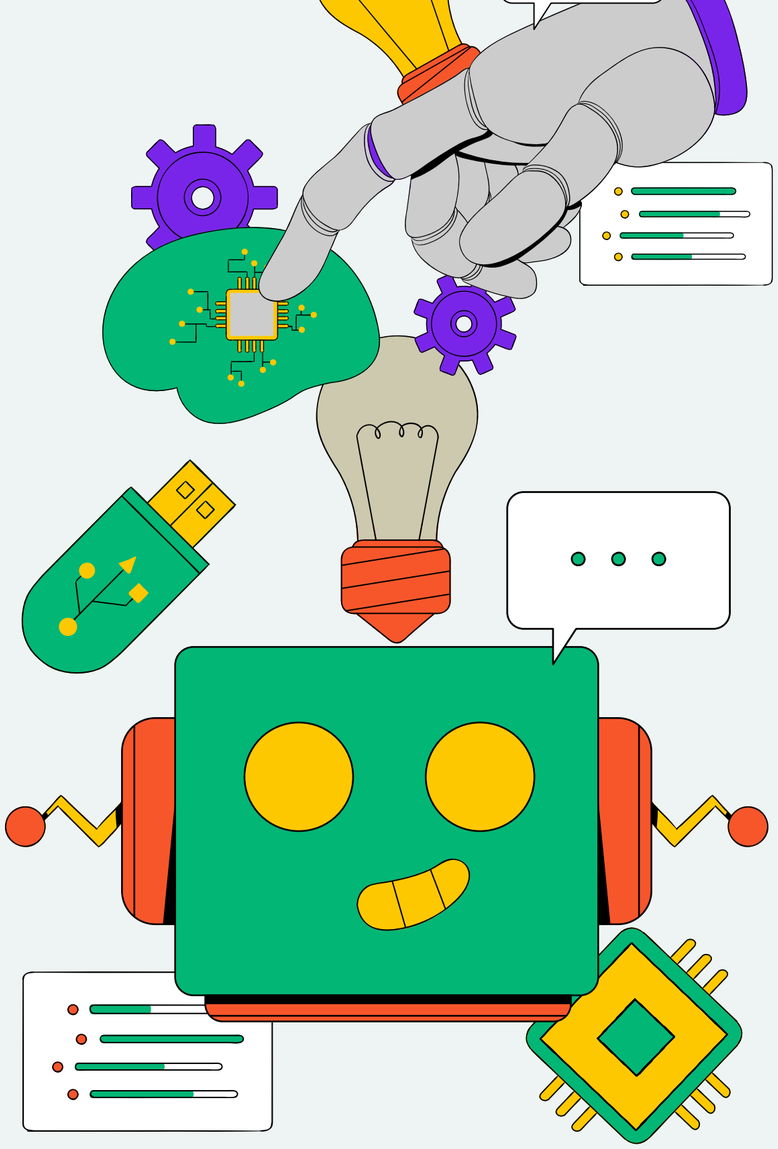
EVALUATION

PID CONTROL



BLUE → INPUT
ORANGE → FEEDBACK
GREEN → ERROR

RMSE = 56



LIMITATION

01

SIGNAL DELAY

There may be delays in reading the potentiometer values or in transmitting control signals to the motor driver when using PPO control.

02

PROCESSING TIME

The microcontroller or processing unit takes a small but non-negligible amount of time to read inputs, compute control actions, and send signals to the motor.

SOLUTION AND FUTURE WORK

SOLUTIONS

- A PID controller, carefully tuned to reduce overshoot and improve response time.
- A Reinforcement Learning (RL) controller, which learns optimal control policies through trial and error and adapts better to system delays.

FUTURE WORK

- GUI or Remote Monitoring
- Hardware Optimization
  - Use Raspberry Pi with available GPIO pins.
- Steer by wire application in vehicle

THANK YOU!